RESTing-LLAMA: Large Language Model based REST API Fuzzing

Varun Gadey
University of Duisburg-Essen
Essen, Germany
mavulliahnagavenkata.gadey@uni-due.de

Christoph Sendner
University of Würzburg
Würzburg, Germany
christoph.sendner@uni-wuerzburg.de

Keven Zimmermann
University of Würzburg
Würzburg, Germany
keven1998@web.de

Alexandra Dmitrienko
University of Duisburg-Essen
Essen, Germany
alexandra.dmitrienko@uni-due.de

Abstract

Recent advances in Large Language Models (LLMs) have introduced new opportunities in software engineering, cybersecurity, and automated testing. Despite this progress, their application to security fuzzing, particularly for REST APIs, remains largely underexplored. Fuzzing remains a fundamental technique for discovering software vulnerabilities through malformed inputs, but traditional fuzzers often struggle to interact effectively with modern REST APIs due to a lack of semantic understanding needed to generate meaningful request sequences. Yet, existing automated fuzzing approaches frequently fail to uncover vulnerabilities due to their inability to use the APIs correctly. We introduce RESTing-LLAMA, a fully automated and end-to-end fuzzing framework that operates in a black-box setting and leverages LLMs to guide the fuzzing process. By extracting semantic information from OpenAPI specifications and natural language documentation, RESTing-LLAMA generates meaningful and well-structured input sequences that align with real-world API behavior. In our evaluation on fifteen real-world APIs, RESTing-LLAMA uncovered 11 vulnerabilities, including one previously unknown vulnerability, while matching or outperforming state-of-the-art fuzzers. It achieved a 34% false positive rate compared to 45% for the next best fuzzer, while requiring 4.4x fewer requests, highlighting the significant improvement in input quality.

CCS Concepts

• **Security and privacy → Web application security**; • **Computing methodologies → Natural language processing**.

Keywords

REST API Fuzzing, Large Language Models, Web Application Security, Automated Vulnerability Discovery

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASIA CCS '26, Bangalore, India
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2356-8/26/06
https://doi.org/10.1145/3779208.3785383

ACM Reference Format:
Varun Gadey, Christoph Sendner, Keven Zimmermann, and Alexandra Dmitrienko. 2026. RESTing-LLAMA: Large Language Model based REST API Fuzzing. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26), June 1–5, 2026, Bangalore, India.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3779208.3785383

1 Introduction

The rapid growth of Artificial Intelligence (AI) and Large Language Models (LLMs) such as GPT-4 [44] and LLaMa-3.3 [56] has transformed multiple domains, including software engineering [26], cybersecurity [39], and quality assurance [32]. These models, trained on massive text corpora, exhibit advanced capabilities in natural language understanding and reasoning. As a result, researchers and practitioners have begun exploring how LLMs can assist with traditionally hard problems, such as identifying software vulnerabilities [48] or automating code analysis [15].

Meanwhile, REST APIs [49] have become the backbone of modern web applications. Statistically, REST remains the most widely adopted API architecture, used by 86% of professionals, and approximately 70% of all public APIs are RESTful [8, 49]. They enable seamless communication between distributed systems, services, and platforms ranging from mobile apps to cloud-based infrastructures. Given the growing usage of REST APIs (21% more traffic year-over-year in 2021 [38]), ensuring their security is essential. REST APIs are increasingly targeted by adversaries, and vulnerabilities in these interfaces have led to numerous high-profile security breaches and data exposures [47, 51]. As a result, ensuring the security and correctness of APIs has become a critical challenge.

Fuzzing remains one of the most effective techniques for uncovering unexpected behaviors and security flaws. However, traditional fuzzers [27, 46] often fall short in the context of REST APIs. Tools like RESTler [12], RESTest [34], and Morest [31] typically generate inputs either randomly, via grammar-based strategies, or through manually crafted heuristics. Despite their sophistication, these approaches often lack a deeper understanding of the API's semantics or its functional dependencies. This leads to low coverage, missed vulnerabilities, and excessive false positives.

A major reason for this gap is that REST APIs typically expose complex behavior through interdependent endpoints, where correct usage requires calling specific endpoints in the right sequence with semantically valid data. These usage patterns are often described

informally in documentation or buried in OpenAPI specifications, which existing fuzzers struggle to interpret. As a result, key vulnerabilities, such as Server-Side Request Forgery (SSRF), which require understanding indirect input flows and internal service interactions, often remain undetected. Even recent efforts like VoAPI² [22], while leveraging functional analysis, still rely on manual validation steps for confirming complex vulnerabilities. In this work, we explore how LLMs can bridge this gap through a fully automated fuzzing framework.

Recent LLM-based tools have begun addressing some of these gaps by using foundation models to generate API requests, validate logic, or infer input formats [29, 53, 60]. However, most focus on functional correctness or testing logic violations and do not explicitly target security vulnerabilities. Moreover, these approaches often operate in isolation, without incorporating end-to-end fuzzing workflows or semantic guidance across API state transitions.

To address this gap, we introduce **RESTing-LLAMA**–a LLM-based fuzzing framework for fully automated, end-to-end testing of REST APIs. RESTing-LLAMA uses LLMs to understand and extract useful semantic information from OpenAPI specifications and documentation, which traditional fuzzers often ignore. This allows our system to generate meaningful request sequences that follow real-world usage patterns and respect the dependencies between API endpoints. RESTing-LLAMA operates in a black-box setting, without requiring access to source code or internal application states. The LLM guides the fuzzing process by identifying logical links between endpoints, suggests likely input formats, and synthesizes request sequences that are more likely to trigger meaningful responses or uncover vulnerabilities.

We evaluate RESTing-LLAMA on eight real-world APIs and compare it with six existing fuzzers. Our experiments show that RESTing-LLAMA achieves exceptional reduction in false positive rate, able to detect both known and unknown vulnerabilities with fewer requests, demonstrating the benefits of combining LLMs with automated fuzzing.

Contributions: In more detail, we make the following contributions:

- We develop RESTing-LLAMA, **the first fully automated, end-to-end fuzzing framework** for REST APIs, that leverages LLMs to extract and utilize semantic information from OpenAPI specifications and documentation in a black-box setting.
- RESTing-LLAMA uncovered **11** vulnerabilities in the 15 real-world REST APIs, including **a zero-day** vulnerability, demonstrating its ability to reveal both known and unknown vulnerabilities.
- Compared to six state-of-the-art fuzzers, RESTing-LLAMA achieves the lowest false positive rate of **34%**, reducing it by 11 percentage points over the next-best tool.
- RESTing-LLAMA requires up to **4.4×** fewer requests to trigger failures, demonstrating its efficiency in generating high-quality test inputs.
- RESTing-LLAMA triggers HTTP 500 status codes at least **15.7x** more often per request, showcasing its superior exploration of edge cases.

Outline. Section 2 introduces the necessary background. Section 3 presents the motivation, threat model, and key challenges. Section 4 details the design of RESTing-LLAMA. Section 5 reports our experimental results. Section 6 reviews related work. Section 7 concludes the paper.

2 Background

This section introduces the foundational concepts essential for understanding our approach.

REST API fuzzing for Vulnerabilities REST (Representational State Transfer [59]) is a widely adopted architectural style for building networked applications, particularly over HTTP. RESTful APIs define operations on resources using standard HTTP methods such as GET, POST, PUT, and DELETE, with each request carrying all the necessary context due to the stateless design. These APIs are favored for their simplicity, scalability, and predictable structure, which supports automation and integration across diverse platforms. However, the same structural consistency also exposes a uniform attack surface. Numerous vulnerabilities such as Server-Side Request Forgery (SSRF) [23], cross-site scripting [30], path traversal [13], and command injection [61], have been documented in REST APIs and cataloged in public vulnerability databases like CVE. Most existing tools test API functionality but neglect security aspects. Fuzzing techniques systematically alter parameters, headers, and payloads to expose flaws that are often missed through conventional testing.

OpenAPI Documentation Format The OpenAPI Specification [55], formerly known as Swagger, is a standardized format for describing RESTful APIs using YAML or JSON. It defines key components such as paths, methods, input parameters, request bodies, and expected responses, all structured in a machine-readable schema. This structured representation allows tools to extract endpoint behavior, validate request syntax, and enforce parameter constraints. Modern frameworks often auto-generate OpenAPI specifications from code, making them both comprehensive and up-to-date. In the context of security testing, the format provides an actionable blueprint for generating semantically valid and diverse API requests, making it well-suited for specification-guided fuzzing.

Large Language Models Large Language Models (LLMs) are advanced AI models trained on extensive text corpora to understand and generate human-like language. By learning to predict words based on preceding context, they capture sophisticated language patterns, including grammar, semantics, and discourse flow. Unlike traditional rule-based systems, LLMs can produce diverse, context-sensitive responses, drawing from their exposure to a wide range of real-world content. Beyond text generation, they can interpret existing text summarizing documents, extracting key points, or identifying semantics, making them valuable in applications where nuanced language understanding is essential.

3 Motivation, Threat Model and Challenges

This section discusses the motivation example of requiring semantic awareness to the fuzzers, defines the assumed threat model, and highlights the key challenges to designing RESTing-LLAMA fuzzer.

3.1 Motivation Example

Figure 1 presents a simplified version of the OpenAPI specification for the `/api/baskets/{name}` endpoint, part of the RBaskets [16] service, a lightweight HTTP request capturing tool designed for debugging and testing webhooks. This endpoint supports a `POST` operation that allows users to create a new basket identified by a path parameter `name` (Line 6). Additionally, the endpoint accepts a structured `requestBody` (Line 12) in JSON format containing several fields such as `forward_url`, `proxy_response`, and `insecure_tls` (Lines 17-25), which influence how the server forwards incoming requests.

A security analysis of this specification revealed that the `forward_url` parameter accepts an arbitrary URL without sufficient sanitization or validation. Using this insight, our semantically aligned fuzzer systematically generated HTTP POST requests where the `forward_url` was populated with internal addresses (e.g., http://127.0.0.1:8080/). The backend logic processed these inputs and issued outbound requests to the attacker-supplied destinations. This behavior constitutes a Server-Side Request Forgery (SSRF) vulnerability, formally registered as **CVE-2023-27163 [40]**. The root cause stems from the absence of access control checks or filtering mechanisms in the component responsible for HTTP forwarding. Since the `forward_url` is treated as a trusted field despite originating from untrusted input, the server violates its trust boundary assumptions, making it easier for adversaries to interact with otherwise unreachable internal services.

Notably, such vulnerabilities are non-trivial to discover through traditional black box fuzzers, since they require semantic awareness of parameters embedded deep within the request body. Our RESTing-LLAMA's ability to parse OpenAPI specifications and generate contextual payloads was crucial in uncovering this issue automatically.

```
1  /api/baskets/{name}:
2    post:
3      summary:Create new basket
4      description:Creates a new basket with this
   name.
5      parameters:
6        - name:name
7          in:path
8          required:true
9          schema:
10           type:string
11         description:The basket name
12     requestBody:
13       required:true
14       content:
15         application/json:
16           schema:
17             type:object
18             properties:
19               forward_url:
20                 type:string
21                 format:uri
22               proxy_response:
23                 type:boolean
24               insecure_tls:
25                 type:boolean
26     responses:
27       '201'
28         description:Created basket
29       '400'
30         description:Malformed request
31       '409'
32         description:Basket already exists
```

Figure 1: API Specification of an endpoint

3.2 Threat Model

In the context of REST API fuzzing, the threat model assumes a black-box setting in which the adversary has no access to the underlying code or internal system state and can only interact with publicly exposed API endpoints. The attacker lacks internal knowledge of the application but seeks to trigger unintended behavior through crafted inputs. Key attack vectors include injection attacks, insecure direct object references, server-side request forgery, and insecure deserialization–issues typically arising from inadequate input validation or improper access controls. The adversary leverages fuzzing to uncover latent vulnerabilities, with potential goals including privilege escalation, unauthorized data access, remote code execution, or denial of service. Beyond the black-box constraint, the attacker is assumed to face no further limitations in capability.

3.3 Challenges

In developing a reliable framework for integrating LLM into the REST API fuzzing, several challenges emerge that must be addressed including:

CH I – Semi-Structured OpenAPI Documentation: OpenAPI specifications mix structured schema with natural language text fields like summaries and descriptions. While the structured parts including endpoints, request parameters, responses can be parsed directly, the unstructured text such as summary, description, and custom metadata often carries critical details, mainly intended behavior or security expectations, that are inconsistently written and harder to interpret. This hybrid format poses a challenge for designing fully automated tools, which must combine parsing with natural language understanding to fully extract meaningful test guidance.

CH II – Execution Bottlenecks from LLM Integration: Fuzzing involves a range of steps from analyzing API behavior to generating and executing large volumes of test cases, all optimized for high-throughput and low-latency. However, to fully leverage API documentation and understand dependencies between endpoints, integrating LLMs becomes essential. From functionality analysis to test case generation, several fuzzing steps benefit semantic reasoning, where LLMs provide valuable guidance. Yet, LLMs introduce significant computational overhead and slower response times compared to traditional approaches. Designing a fully automated fuzzer with LLM integration, without significantly affecting runtime, is a core challenge.

CH III - Prompt Sensitivity Across Fuzzing Stages: LLMs are inherently sensitive to the phrasing, structure, and completeness of their input prompts. In a multi-stage fuzzing pipeline, each stage including dependency analysis, functionality analysis, or input generation, requires distinct outputs from the LLM. This variability in prompt response behavior introduces a challenge: Small differences in input can produce inconsistent or irrelevant responses, which may affect the fuzzer's accuracy, coverage, and runtime performance. Designing effective and consistent prompts that match the intent of each fuzzing stage is a non-trivial challenge.

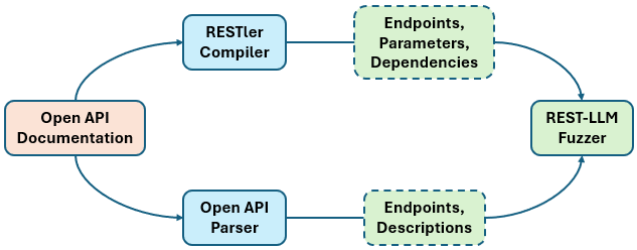


Figure 2: API Parsing

4 Design of RESTing-LLAMA

We propose a hybrid REST API fuzzing framework that leverages LLM's to interpret and utilize unused human-readable information available in OpenAPI documentations. While existing state-of-the-art fuzzers [12, 27, 46] are capable to automate various testing processes, they are limited in the amount of documentation they can ingest and utilize for fuzzing. They are typically only able to process a small portion of this information in a structured format, and fail to process and make complete use of the available rich natural language descriptions that provide deeper semantic cues of how an API functions. Our approach incorporates LLM processing that interprets such human-readable content to guide and augment the fuzzing pipeline. This strategic way to leverage LLMs in a fuzzing pipeline makes the better use of the API documentation to improve the quality and effectiveness of the fuzzing campaign.

Our approach enables us to take advantage of the distinct strengths of both, traditional and LLM-based approaches. While LLMs interpret natural language descriptions to uncover semantic dependencies and behavioral cues, conventional parsing techniques remain effective for directly parsing structured components such as endpoint paths, parameter types, and response formats. By combining these complementary methods, our framework covers more of the API specification while maintaining the speed and reliability expected from effective fuzzing.

Target APIs: Our fuzzer is designed specifically for REST APIs that adhere to the architectural principles of the REST standard, including the use of conventional HTTP methods. It operates in a black-box manner, requiring no access to the source code or internal runtime state. Instead, it interacts with the deployed API either locally or remotely by issuing HTTP requests and analyzing the resulting responses to detect vulnerabilities.

The only prerequisite is that the API is documented in the OpenAPI format, which is the de facto standard for describing REST APIs. In our design, this documentation serves as the sole source of information for the fuzzer and must be accurate and comprehensive. In particular, human-readable elements — such as endpoint descriptions and parameter explanations — play a critical role, as they often convey semantic intent, inter-parameter dependencies, and security considerations.

API Parsing: To support our hybrid fuzzing strategy, we formulate a dual-path parsing workflow tailored to extract both structured and semantic information from OpenAPI documentation. Figure 2 illustrates this workflow: Structured elements such as endpoints, parameters, and inter-endpoint dependencies are extracted using the RESTler compiler from RESTler [37], while our OpenAPI parser focuses on human-readable descriptions and semantic cues. These complementary components produce distinct but synergistic outputs that serve as input to the RESTing-LLAMA fuzzer. This combined parsing strategy addresses the challenges introduced by the semi-structured nature of OpenAPI specifications (CH1), ensuring both the syntactic correctness and semantic relevance of generated test cases.

Fuzzing Pipeline: Figure 3 provides an overview of the pipeline architecture, highlighting the interactions between parsing steps, fuzzing steps, the LLM module, and the target API. At the core of our RESTing-LLAMA is a pipeline-based architecture that allows flexible information flow between the different steps of the fuzzing process. In this architecture, the LLM is tightly integrated and interacts with all the steps of the fuzzer, and no longer strictly linear. Unlike traditional fuzzers [12, 46, 58], where the information flow is strictly linear, our design allows each stage in the pipeline to query the LLM and receive additional context at any point during execution. This architectural flexibility allows the system to adapt and improve based on intermediate results, facilitating more thorough and intelligent testing of REST APIs.

Communication between LLM and Fuzzing Pipeline The LLM runs independently of the fuzzer, ensuring that it does not introduce significant overhead or latency into the fuzzing pipeline. At every stage of the fuzzing process, the fuzzer can use the LLM to extract information. This approach provides a flexible strategy where the LLM can dynamically contribute where needed. However, because the fuzzing stages are running concurrently, the LLM is shared with limited computational resources. Therefore, concurrent requests must be limited. To achieve this, we employ a request queue that ensures only a certain amount of prompts are processed by the LLM at a time, as seen in fig. 4, preventing system overload and ensuring the fuzzer remains efficient.

The response from the LLM is handled by the response interpreter, which parses the output from the LLM and puts it back into the appropriate stage of the fuzzing pipeline. This architecture allows stages of the fuzzing process to interact easily with the LLM. The design also enables every stage of the fuzzing pipeline to contribute to the fuzzing process equally, as every stage can send prompts to the LLM, which can be interpreted and reinserted into every stage there is.

High-level overview: The fuzzing process begins with API parsing and compilation, where OpenAPI documentation is analyzed to extract both structured and human-readable information. The pipeline

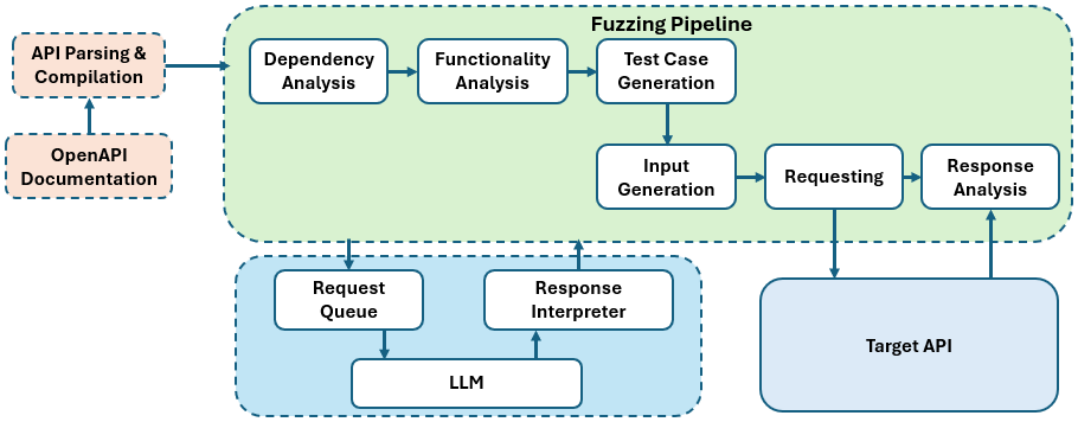


Figure 3: RESTing-LLAMA's Fuzzing Pipeline

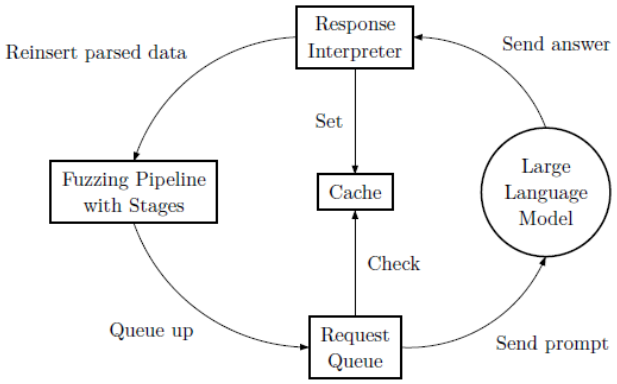


Figure 4: Communcation between LLM and the Fuzzing Pipeline

proceeds with dependency analysis, where it resolves any dependencies between API endpoints to be able to create valid request sequences. Next, during functionality analysis, the RESTing-LLAMA fuzzer guesses the functionalities associated with every request parameter. All of this information is combined in the test case generation stage to create test cases for every identified functionality. Missing inputs are generated for every test case in the input generation stage. The requesting stage executes every test case against the targeted API, sending the received responses to the next stage. Finally, the response analysis component checks whether the responses received indicate the existence of any vulnerabilities. In the following, we discuss individual stages in the pipeline in more detail.

4.1 Dependency Analysis

In REST API fuzzing, dependencies between endpoints refer to relationships where the request parameters of an endpoint depend on the response parameters of another endpoint. Such dependencies occur when an API feature requires multiple sequential calls to function correctly. Consider, for example, an API where users must be created before creating some objects for them. A **POST /users** endpoint may return a response containing a **userId**, which is subsequently required as a request parameter for a **POST /users/user-id/objects** endpoint. Without being aware about this dependency, a traditional fuzzer would not be able to generate valid request sequences and might fail to explore some parts of the API's functionality.

Although traditional fuzzers rely on structural analysis and can detect some of these dependencies based on parameter names and endpoint paths, they are largely limited to the cases where clear naming conventions are set. Notably, they lack ability to find dependencies between parameters with differing naming conventions, implicit relationships, or undocumented behaviors. This is exactly what our RESTing-LLAMA achieves in dependencies analysis step: With integration of LLMs, we process natural language descriptions and names to identify dependencies that would otherwise remain undetected. This combination of structural and LLM-based analysis ensures that the fuzzer can generate valid request sequences. By understanding discovered dependencies, we can determine which endpoints need to be called first to make further requests work seamlessly. In the user creation example we mentioned above, the fuzzer must first send a request to **POST /users**, get the userId from the response, and then use this value when calling **POST /users/userId/objects**. Without this information, requests might fail because of missing or invalid parameter values.

The carefully crafted prompt for LLM for this dependency analysis step is provided in Appendix A Since LLMs are inherently sensitive to the phrasing, structure, and completeness of their input prompts, this stage-specific prompt introduces variability that must be handled carefully (CH3).

4.2 Functionality Analysis

The functionality analysis stage plays a key role in identifying how different request parameters are used within the API. By categorizing parameters based on their potential functionalities, the fuzzer can craft targeted test cases aimed at specific vulnerability classes including SSRF and Path Traversal. Without this analysis, test generation would be limited to generic payloads, reducing the effectiveness of the fuzzing process. Each parameter may serve multiple purposes simultaneously e.g., a parameter specifying a file

path on the server might also be reflected in the server's response, making it relevant for both Path Traversal and XSS detection. To account for such cases, our approach supports the association of a single parameter with multiple functionalities. This broader functional mapping increases test coverage and improves the likelihood of discovering security flaws.

We consider the following set of functionalities: (i) Proxy, (ii) Path, (iii) Command, and (iv) Display. **Proxy Functionality** includes parameters that hold URLs or host names that may be used by the API to fetch remote resources. **Path Functionality** holds parameters that contain file or directory paths and are often used by the backend to access or manipulate files on the server. **Command Functionality** comprises parameters that are used within shell commands on the server and pose a risk of Command Injection [61]. **Display Functionality** includes parameters that are directly reflected in the server's response and could be exploited for XSS attacks.

Just like in the dependency analysis step, here we determine the functionalities of all the parameters with combination of both structural and LLM based analysis. We start with a structural analysis, which is based on predefined lists of inherited keywords from [21]. For each functionality, we maintain two word lists: One for parameter names and one for endpoint paths. If a parameter name contains a keyword from the name-based word list, it is assigned that functionality. Similarly, if an endpoint path contains a keyword from the path-based word list, all parameters of that endpoint inherit the functionality. However, as parameter names can be arbitrary and may not always follow a predictable naming convention, the structural analysis alone is not sufficient. With the integration of LLM driven analysis, we find out more different functionalities through endpoint descriptions than through explicit naming patterns. For example, consider an endpoint description stating: **This parameter specifies the URL for webhook notifications**. It clearly indicates that the parameter is used for proxying requests, yet its name might not reflect this purpose. After performing both, structural and LLM-based analysis, the fuzzer serves the merged result as input to the next stage. We provide a sample prompt template handcrafted for this step in Appendix A.

4.3 Test Case Generation

In this step, the RESTing-LLAMA creates test cases by using both, the identified parameter functionalities and the dependency relationships between endpoints. For each parameter that has an assigned functionality, the RESTing-LLAMA first identifies the target request and then builds a valid request sequence by tracing its dependencies. These requests from the sequence must be sent in the correct order to ensure that each one returns the values needed to fill in the parameters of the next. This helps the fuzzer build valid request sequences that match the expected behavior of the API.

Test case generation follows two complementary paths. First, the RESTing-LLAMA constructs test cases directly from the extracted functionality and dependency data, without involving the LLM. For every parameter and each of its assigned functionalities, a separate test case is generated, ensuring broad coverage across different vulnerability types. Each test case includes a request sequence with templates containing placeholders, which will be filled later during input generation. Secondly, the LLM is asked to generate additional test case templates based on the API documentation and the specific type of vulnerability being tested. To keep the LLM focused on generating vulnerability specific logic, it is explicitly instructed to exclude dependency requests from its output. This hybrid strategy ensures that both known test patterns and context-rich variations are explored. Figure 5 illustrates a sample test case composed of sequential request templates with dependencies and unresolved values. For example, the third request requires IDs returned by the first two requests. Parameters labeled as *placeholder* are filled during input generation, while those marked as *dependent* are resolved at runtime from earlier responses.

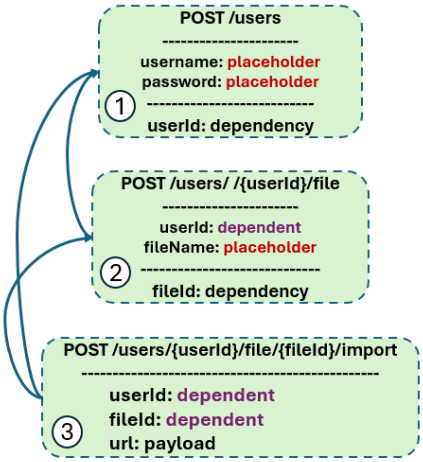


Figure 5: Test case Example

To support broader security testing, the RESTing-LLAMA also prompts the LLM to produce test cases that are not tied to specific parameter functionalities, particularly for scenarios involving authentication and access control. These include tests with multiple users or role-based interactions, which are essential for exposing authorization flaws and business logic vulnerabilities. The resulting test cases, consisting of request sequences with parameter placeholders, are passed to the input generation stage.

4.4 Input Generation

At this step, the RESTing-LLAMA fills in the remaining placeholder values in the test cases generated earlier. These placeholders represent parameters that are not tied to dependency relationships or predefined payloads. Although some values such as those derived from functionality types may already be present in the templates, many parameters remain unset and must be completed here. Input generation focuses on producing concrete values for these open fields.

The fuzzer uses parameter type information extracted from the OpenAPI specification via the RESTler compiler to select or generate suitable input. These types follow standard JSON schema formats, such as strings, integers, booleans, dates, array, object, and may also include more complex types like date, time, or datetime. For each supported type, RESTing-LLAMA uses internal generators to produce appropriate values. Strategically, we limit the use of

LLMs in this step and rely on standard input generators to mitigate the computational overhead during runtime (CH2). To maintain consistency while generating values across request sequences, parameters with the same name are assigned the same generated value. This is important, as we do not track dependencies between request parameters, which is necessary for some APIs. Consider an API with an endpoint to create objects called **POST /objects/-name** with the associated endpoint to retrieve the objects called **GET /objects/name**. Obviously, we want to use the same value for the parameter name to be able to use these endpoints in the same test case. We would lose the ability to retrieve the created object if we generate a unique value for this parameter every time. After this stage, the only remaining values to left to be filled are those dependent on earlier responses. These are resolved dynamically during the requesting phase, where the test cases are sent to the target API.

4.5 Requesting

This step is responsible for executing the test cases by sending HTTP requests to the target API. It takes as input the finalized request templates from the input generation stage, resolves remaining dependencies using previous responses, and sends the requests in the correct sequence to preserve the logical flow of each test case. Each request is constructed by placing parameter values into their correct locations, such as the URL path, query string, or request body. These positions are determined based on the OpenAPI specification, and RESTing-LLAMA ensures that all values are properly formatted, encoded, and escaped to prevent malformed requests. While request headers are not explicitly fuzzed, the system attaches required headers and any provided authentication credentials to enable valid communication with the API. All requests in a test case are executed sequentially to satisfy dependency constraints, allowing later requests to use values returned by earlier ones. During this process, RESTing-LLAMA also tracks metadata such as success rates and failure cases for later analysis.

The LLM is not involved in this step, as the execution logic is purely procedural and does not benefit from LLM-based reasoning or decision-making, which would only introduce unnecessary delays in processing each request. Once all requests in a test case are executed, the collected responses are passed to the next step to find targeted vulnerabilities.

4.6 Response Analysis

This final step analyses the responses received from the API to determine whether a targeted vulnerability was successfully triggered. For each test case, the fuzzer applies detection logic specific to the targeted vulnerability type, using patterns, response content, or external signals to classify outcomes. RESTing-LLAMA employs traditional oracle based checks [12, 21] for response behavior and status codes, augmented by a validation server for vulnerabilities such as SSRF, stored XSS, and command injection. This server monitors whether injected payloads led to outbound connections or script execution, providing clear indicators of exploitation.

To detect vulnerabilities like path traversal and XSS, RESTing-LLAMA inspects server responses for signs of payload reflection (Appendix D) or sensitive data leakage. For broken access control, the system analyzes whether restricted actions succeed when performed by unauthorized or unauthenticated users, based on status codes and response content. Across all cases, detection remains lightweight and efficient, without involving the LLM at this stage. The results are used to evaluate test case effectiveness and flag potentially vulnerable API behaviors.

Server-Side Request Forgery Server-Side Request Forgery (SSRF) vulnerabilities allow attackers to force a server to issue requests to unintended locations. These can be exploited to access internal services or retrieve private data. Such vulnerabilities occur when user input is used to fetch external resources without proper validation. Attackers can craft inputs that direct the server to access internal networks and services protected by firewalls, such as cloud metadata services. This may further lead to exploitation of vulnerabilities in internal applications. For example, gaining access to an internal database server. Fundamentally, the exploited issue in vulnerable applications is the ability of the user to control the application's requests to perform unintended actions. Accepted parameters may be embedded into URLs on the server side. To mitigate this, resulting URLs should be validated against a strict whitelist of accessible networks. After constructing the URL, the server should resolve the domain name and verify that it belongs to an approved network.

During the request, the server must also prevent HTTP redirects, which could otherwise be used to access internal applications. Figure 6 shows a sequence diagram of a correct protection mechanism against SSRF attacks as described above.

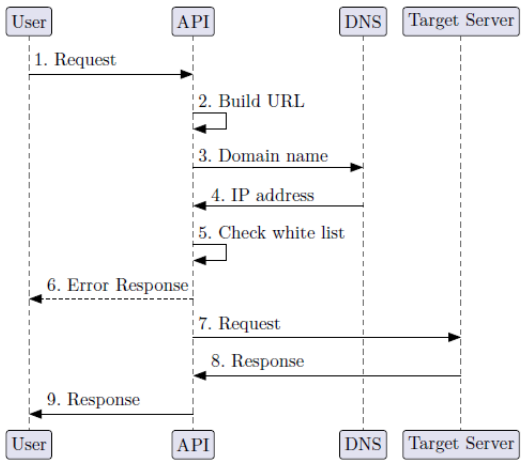


Figure 6: Sequence diagram of a correct protection mechanism against SSRF attacks

Testing for SSRF generally involves crafting requests to endpoints that trigger external requests. The goal is to manipulate these requests to reveal SSRF vulnerabilities. In our approach, we use an external server for request detection. Any requests received by this server are logged and reported back to the fuzzer. We inject test payloads containing URLs pointing to this server, each embedded with a unique identifier for the test case. This allows the fuzzer to identify which payload triggered the request.

As the server is located outside of the local network of the targeted application, a detected request does not proof an SSRF vulnerability. Whether the vulnerability exists or not, must be tested manually. This method ensures that even remote servers can be tested for SSRF vulnerabilities by the fuzzer, even if its not possible to get the verification server to run on the same network as the target API. Although this introduces some manual confirmation effort, features that are potentially vulnerable to SSRF attacks are rather rare. Therefore, the time needed is comparable with just checking for false positives of other detected vulnerabilities.

For detailed detection strategies corresponding to Cross-Site Scripting, command injection, and path traversal, please refer to Appendix D.

5 Evaluation

This section evaluates RESTing-LLAMA's effectiveness in detecting vulnerabilities across a diverse set of real-world REST APIs. We begin by detailing our experimental setup, including the tested applications, baseline tools, LLM configurations, and evaluation metrics. We then compare the effectiveness of RESTing-LLAMA in vulnerability discovery with the state-of-the-art fuzzers. This includes a breakdown of the zero-day and known vulnerabilities identified, comparisons across different LLMs, and results contextualized by API endpoint coverage. We further evaluate the false positive rate, runtime efficiency, and the proportion of server errors triggered (HTTP 500), highlighting where RESTing-LLAMA performs particularly well. Finally, we analyze RESTing-LLAMA's runtime efficiency, and the impact of different LLMs integrated into the fuzzing pipeline.

5.1 Experimental Setup

5.1.1 Compared Tools. We compare RESTing-LLAMA against both, general-purpose vulnerability scanners and REST API-specific fuzzers, to highlight its effectiveness and coverage. The following tools are chosen to cover different fuzzing strategies and detection capabilities:

- **ZAP** [46]: A black-box vulnerability scanner widely used for web penetration testing. We use ZAP's API scanner mode.
- **Astra** [27]: A specialized REST API vulnerability scanner that mutates parameters and headers to detect common weaknesses.
- **RESTler** [12]: A black-box REST API fuzzer from Microsoft that infers request dependencies and generates stateful test sequences.
- **RestTestGen** [58]: A data-dependency-aware fuzzer that generates sequences using inferred request relationships.
- **MINER** [33]: A neural sequence generator that uses LSTM-based prediction and length-oriented strategies to generate deep API request chains.
- **VoAPI^2** [22]: A vulnerability-oriented REST API fuzzer that uses parameter functionality inference to guide test case construction and improves detection coverage through customized oracles.

5.1.2 Compared LLMs. To evaluate the impact of different language models on the fuzzing pipeline, we integrate four LLMs representing a range of sizes and design goals. Table 1 provides the list of LLMs with their parameters count and context size details considered for the evaluation. These LLMs vary in terms of architecture, training data, and target use cases. Some, like Claude 3.7, are tuned for programming tasks, while others prioritize general-purpose tasks.

Table 1: LLMs used in evaluation

LLM	Param Count	Context Size	Max Output Tokens
Llama 3.3 [56]	70B	20K	Unlimited
GPT-4o mini [44]	Unknown	128K	16,384
DeepSeek-V3 [17]	67B	128K	8,000
Claude 3.7 Sonnet [9]	Unknown	200K	64,000

5.1.3 Target Applications. We select a diverse set of real-world open-source REST APIs across application domains and scales. All selected applications have publicly available OpenAPI specifications and Docker images, making them suitable for automated fuzzing. To ensure broad coverage, we include applications that meet these criteria from popular self-hosted software repositories. Based on the number of documented endpoints, we also categorize them into small, medium, and large-scale applications and provide an evaluation of how our RESTing-LLAMA performs on varying sizes. The applications that are consistently evaluated by all the benchmark tools were also added in the list,

Table 2 showcases the list of all the real-world REST API applications that vary in size, domain, and downloads are considered for the evaluation. All the applications that are consistently evaluated by the benchmark tools were also considered and included in the list for a fair comparison.

Table 2: Benchmark Applications

Application	Version	#Endpoints	Downloads	Domain	Scale
Answer [10]	1.4.5	173	288K	Q&A Platform	Medium
Appwrite [11]	0.9.3	95	9.7M	Backend Platform	Medium
Audiobookshelf [1]	2.16.0	65	2.9M	Media Library	Small
Casdoor [2]	1.13.0	68	523K	Auth Server	Small
Chatwoot [3]	4.0.4	166	5M	Customer Support	Medium
Commafeed [4]	5.5.0	43	162K	RSS Reader	Small
Ever Traduora [5]	0.20.2	49	50K	Translation Tool	Small
Gitea [6]	1.16.7	291	353M	Code Hosting	Large
Grocy [14]	4.5.0	86	55M	Household Management	Medium
Jellyfin [7]	10.7.1	410	283M	Media Server	Large
Kavita [28]	0.8.5.11	360	185K	Ebook Library	Large
Komga [25]	1.21.1	230	34M	Comic Reader	Large
Microcks [36]	1.7.1	79	660K	API Mocking	Small
Outline [45]	0.82.0	78	9M	Team Wiki	Small
Rbaskets [16]	1.2.3	21	324K	Snippet Hosting	Small

5.1.4 Evaluation Metrics. We use the following metrics to evaluate the performance and quality of fuzzing.

- **Vulnerability Count:** Number of unique vulnerabilities detected automatically using in-built oracles.
- **False Positive Rate (FPR):** Fraction of reported issues that were not true vulnerabilities, validated via manual inspection.
- **Coverage:** Proportion of API endpoints successfully exercised by valid requests (status code 200–399).
- **Runtime:** Time taken to fuzz each API, both with and without cached LLM outputs.

- **500 Ratio:** Fraction of responses returning HTTP 500 status codes, indicating potentially unhandled errors.

5.1.5 Test Setup. All APIs were deployed in isolated containers using Docker Compose. If an API required setup steps (e.g., user account creation, API key provisioning), we automated these via a Python script. The script also executed the fuzzer, captured metrics, and ensured consistent teardown of containers after each run.

For fair comparison, each fuzzer was run against the same OpenAPI specification and testbed. Any invalid or incompatible specifications were manually corrected where possible. When models or tools required specific input formatting or network setup, appropriate adapters were implemented. All tests were repeated across a range of tools and LLMs using consistent parameters and time budgets.

5.2 Vulnerability Detection

We evaluate the ability of our RESTing-LLAMA across a wide range of real-world REST APIs. Our evaluation spans 15 open-source applications from various domains and scales, a considerably larger set than, for instance, was evaluated in VOAPI2 [22] and MINER [33]. As summarized in Table 3, RESTing-LLAMA successfully identified 11 vulnerabilities, including all 6 known and 1 previously undocumented (zero-day), 4 found and patched (with no CVE-Id) vulnerabilities across multiple endpoints and in 8 real world APIs. Notably, all discovered vulnerabilities were triggered fully automatically, without requiring manual payload injection or human-in-the-loop filtering. In comparison, while **VOAPI2** [22] and **MINER** [33] also demonstrate strong detection capabilities, they are limited in scope and automation. RESTing-LLAMA detected vulnerabilities in more applications and showed broader endpoint coverage, particularly for **SSRF** vulnerabilities.

In the case of **Appwrite**, RESTing-LLAMA uncovered two distinct SSRF vulnerabilities linked to `CVE-2023-27159 [43]` matching VOAPI2's output while surpassing MINER and RESTler, which failed to detect one or both endpoints. Web scanners like **ZAP** and **Astra** also flagged these vulnerabilities but lacked consistency across other targets. For **Gitea**, ZAP and VOAPI2 are detected one SSRF in the `/repos/{owner}/{repo}/hooks` endpoint (`CVE-2018-15192 [41]`). In **Jellyfin**, RESTing-LLAMA correctly flagged both SSRF endpoints (`/Images/Remote` and `/Items/RemoteSearch/Image`) associated with CVE-2021-29490 [42], while the remaining tools such as RESTler and MINE missed it entirely, demonstrating the advantage of leveraging API documentation into fuzzer over generic fuzzing. Notably, in **Microcks**, although VOAPI2 detected the SSRF in `/jobs`, its detection relies on the inclusion of application specific code associated with the endpoint [21]. Therefore, this vulnerability cannot be considered as having been detected automatically.

In smaller-scale applications such as **Rbaskets**, **Kavita**, and **Commafeed**, RESTing-LLAMA surfaced SSRF vulnerabilities that were not reported by any other tools used in evaluation. For instance, in Rbaskets, our fuzzer not only detected a known SSRF **(`CVE-2023-27163 [40]`)** but also uncovered a new one in the **`/api/Theme/download-theme`** endpoint. Similarly, in Kavita, RESTing-LLAMA flagged two SSRF vulnerabilities that were unexplored by all other tools, suggesting better coverage of file and upload interfaces often overlooked by existing heuristics. The first is a recently patched vulnerability with no assigned CVE ID, while the second is a brand-new zero-day vulnerability discovered by RESTing-LLAMA and reported to the developers of Kavita. These vulnerabilities were found in the **`POST /api/Theme/download-theme`** and **`POST /api/Upload/upload-by-url`** endpoints. Notably, our tool was the only one among the compared tools to identify two SSRF vulnerabilities that were already recently patched in Commafeed. As a small-scale application, Commafeed does not have CVEs assigned to it, further affirming RESTing-LLAMA's semantic reasoning capabilities.

5.2.1 Test Request Volume and Time Efficiency. Table 4 compares the number of HTTP requests (packets) and total testing time across all the benchmark fuzzers. The results include experiments on RESTing-LLAMA and results reported in the VOAPI2 [22]. RESTing-LLAMA issues substantially fewer test requests than all other tools, indicating a highly targeted and efficient exploration strategy. For instance, while VOAPI2 sent over 13,000 requests to Jellyfin, RESTing-LLAMA triggered the same set of vulnerabilities using just ~8,000 requests, a reduction of over 40%. The difference is even more pronounced in larger applications like Appwrite and Gitea, where RESTler and MINER generated hundreds of thousands of packets, yet our RESTing-LLAMA needed only a few hundred to discover the same vulnerabilities, highlighting its effective and efficient coverage without excessive brute-force enumeration.

Despite sending far fewer requests, RESTing-LLAMA's total fuzzing time remains competitive. In fact, for five out of the six applications, our tool completed the analysis faster than or within a comparable timeframe to VOAPI2. For example, RESTing-LLAMA tested Microcks in just 8m36s, whereas VOAPI2 took 24s, a marginal difference considering the request volume gap. On other benchmarks like Gitea, RESTing-LLAMA completed the scan in 50m34s, while VOAPI2 took 7m05s but required nearly three times the number of requests. This illustrates that RESTing-LLAMA prioritizes semantic reasoning and payload placement over sheer volume, making it a more scalable option for large-scale deployments. Additionally, an evaluation comparing baselines under an equal budget is described in Appendix B.

5.2.2 Ratio of 500 Status Code. As shown in Table 5, RESTing-LLAMA triggers a higher ratio of HTTP 500 status codes in certain applications like Casdoor and Microcks. This is a direct result of its semantic-guided approach, which makes deeper exploration of execution paths that often surface unstable or unintended behaviors. Rather than malformed inputs, these errors typically stem from valid but complex test cases. These ratios suggest that RESTing-LLAMA spends more of its time exercising edge cases and triggering backend failures, highlighting its strength in uncovering subtle robustness issues in API logic.

5.2.3 Ablation Study. To understand the contributions of LLM-guided stages, we executed controlled and individual ablations by disabling the LLM in the dependency-analysis and functionality-analysis stages of the RESTing-LLAMA seperately and tested on appwrite. As shown in Table 6, removing the LLM sharply reduced the number of inferred dependencies 49 to 4 and identified functionalities 159 to 7, demonstrating that the LLM extracts semantic

Table 3: Detected REST API vulnerabilities across tools. Each row indicates an API endpoint and the corresponding known or unknown SSRF vulnerability. A ✓ indicates that the tool successfully identified the vulnerability, while a ✗ denotes failure to detect it. *unassigned & patched* label indicates that one vulnerability is found and patched with an unassigned CVE-Id. *unassigned* label indicates zero-day vulnerability has been discovered and reported.

Application	Version	Endpoint	Vulnerability	CVE ID	Vulnerability Identification Tools						
					VOAPI2	RESTler	RestTestGen	MINER	ZAP	Astra	RESTing-LLAMA
Appwrite	0.9.3	GET /avatars/favicon	SSRF	CVE-2023-27159	✓	✗	✗	✗	✓	✓	✓
		GET /avatars/image	SSRF	CVE-2023-27159	✓	✗	✓	✗	✓	✓	✓
Gitea	1.16.7	POST /repos/owner/repo/hooks	SSRF	CVE-2018-15192	✓	✗	✗	✗	✓	✗	✗
Jellyfin	10.7.1	GET /Images/Remote	SSRF	CVE-2021-29490	✓	✓	✗	✓	✓	✓	✓
		GET /Items/RemoteSearch/Image	SSRF	CVE-2021-29490	✓	✗	✗	✗	✗	✓	✓
Microcks	1.7.1	POST /jobs	SSRF	unassigned & patched	✓	✓	✓	✗	✗	✗	✗
Rbaskets	1.2.3	POST /api/baskets/name	SSRF	CVE-2023-27163	✓	✗	✗	✗	✗	✗	✓
		POST /api/Theme/download-theme	SSRF	unassigned & patched	✗	✗	✗	✗	✗	✗	✓
Kavita	0.8.5.11	POST /api/Theme/download-theme	SSRF	unassigned & patched	–	–	–	–	–	–	✓
		POST /api/Upload/upload-by-url	SSRF	unassigned	–	–	–	–	–	–	✓
Audiobookshelf	2.16.0	POST /api/podcasts/feed	SSRF	CVE-2023-51697	✗	✗	✗	✗	✗	✓	✓
Commafeed	5.5.0	POST /rest/feed/fetch	SSRF	unassigned & patched	–	–	–	–	–	–	✓
		POST /rest/feed/subscribe	SSRF	unassigned & patched	–	–	–	–	–	–	✓

Table 4: Comparison of Test Requests and Testing Time across Tools

Tool	Appwrite		Casdoor		Gitea		Jellyfin		Microcks		Rbaskets	
	Packets	Time	Packets	Time	Packets	Time	Packets	Time	Packets	Time	Packets	Time
RESTler	88,558	5h	165,470	5h	297,419	5h	2,175	12m07s	67,128	5h	20,435	5h
MINER	64,944	5h	104,352	5h	31,668	5h	23,002	5h	143,830	5h	17,422	5h
RestTestGen	9,030	19m49s	16,340	34m13s	58,200	4h37m	71,440	4h43m	7,340	8m5s	2,140	1m32s
RestTestGen+V	6,334	12m42s	8,920	13m09s	32,550	1h28m	48,740	3h57m	4,246	5m51s	1,550	1m03s
VOAPI2	2,123	1m51s	6,987	6m27s	9,137	7m05s	13,578	10m53s	509	24s	238	13s
RESTing-LLAMA	481	5m59s	202	3m32s	3,125	50m34s	8,098	6m53s	600	8m36s	399	1m50s

Table 5: Ratio of HTTP 500 Status Codes Encountered Across Fuzzers

Application	MINER	RESTler	RestTestGen	VOAPI2	RESTing-LLAMA
Appwrite	0.006%	0.006%	0.011%	0.047%	0%
Casdoor	0%	0%	0%	0%	16.83%
Gitea	0.003%	0%	0.003%	0%	0%
Microcks	0.009%	0.028%	0.204%	0.393%	17.87%
Jellyfin	0.296%	2.483%	0.088%	0.169%	4.8%
Rbaskets	0%	0%	0%	0%	0%
Average	**0.052%**	**0.42%**	**0.051%**	**0.102%**	**6.585%**

Table 6: Component-level ablation study on Appwrite

Fuzzing Stage	With LLM	Without LLM
Dependencies Discovered	49	4
Functionalities Identified	159	7
SSRF Vulnerabilities Found	2	0

relationships between API endpoints. Notably, table 3 and table 6 the LLM-included configuration in both stages of the tool identified two SSRF vulnerabilities, whereas the No-LLM variant in both ablation studies found none.

5.3 RESTing-LLAMA's Efficiency

Beyond vulnerability detection, an effective API fuzzer must also demonstrate operational efficiency, minimizing false positives while maximizing test coverage across real-world applications. In this section, we evaluate RESTing-LLAMA on these two fronts. First, we analyze its false positive rate compared to other considered benchmark fuzzers. Then, we assess the breadth of API coverage achieved during fuzzing to understand RESTing-LLAMA's exploration capability better.

5.3.1 False Positive Rate. False positives undermine the usefulness of fuzzing tools by overwhelming developers with incorrect or non-actionable vulnerability reports. To measure precision, we compare RESTing-LLAMA against traditional and robust web scanners such as ASTRA and ZAP, as well as the recent promising fuzzer VOAPI2. We compare the evaluation results across six applications that are consistently used by all tools: Appwrite, Casdoor, Gitea, Jellyfin, Microcks, and Rbaskets.

Figure 7 presents the false positive rates across these benchmarks. RESTing-LLAMA consistently yields fewer false positives than all other tools. For instance, it reports 0% false positives on Appwrite and Microcks, while ASTRA and ZAP report up to 20–35% on the same targets. On average, RESTing-LLAMA maintains a

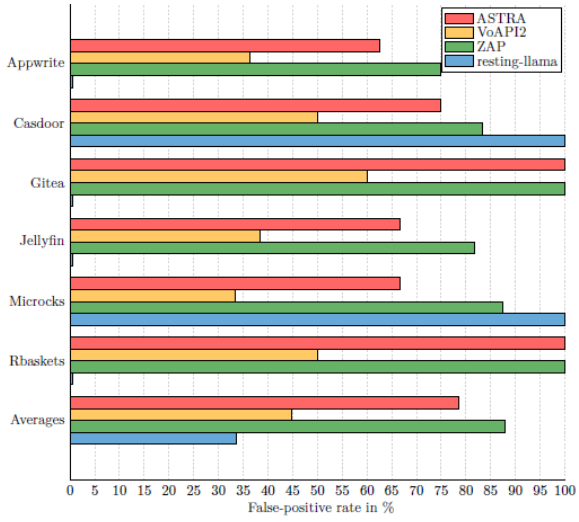


Figure 7: False Positive Rate

false positive rate of only 34%, compared to 45% for VOAPI2, while traditional scanners like ZAP and ASTRA exceed 70%, underscoring RESTing-LLAMA's superior precision across diverse applications. This low false positive rate is attributed to RESTing-LLAMA's semantic-guided testing strategy, which avoids brute-force payload injection and instead generates context-aware requests that align with API behavior.

5.4 Comparison of LLMs

To assess the impact of different LLMs on fuzzing effectiveness, we evaluated multiple LLM configurations integrated into our fuzzing pipeline, alongside a baseline configuration without LLM support. The goal of this analysis is to identify the most effective LLM variant and to quantify the performance gains introduced by LLM-guided fuzzing. Table 7 summarizes the vulnerabilities discovered by each LLM configuration. Across all benchmarks, LLMs consistently improved the number of vulnerabilities detected. On average, even the weakest-performing LLM model identified 11.67% more vulnerabilities than the No-LLM baseline. The strongest performer, Claude 3.7 Sonnet, outperformed the baseline by 27.4%. These improvements stem from the LLMs' ability to semantically interpret API documentation and generate more targeted test cases. For instance, in the **Audiobookshelf** API, the No-LLM configuration failed to uncover a vulnerability due to the lack of explicit proxy-related keywords, whereas all LLMs succeeded by leveraging contextual information in the path and description fields. Similarly, for **Rbaskets**, only Claude 3.7 Sonnet and Llama 3.3 were able to detect hidden vulnerabilities across multiple related endpoints highlighting their ability to resolve implicit dependencies. Interestingly, despite being larger in size, models like DeepSeek-V3 and GPT-4o Mini failed to identify these vulnerabilities, possibly due to differences in training objectives or exposure to relevant security focused data during pretraining.

5.5 Ethical Considerations

We reported the discovered SSRF vulnerability in Kavita to the developers at the time of paper submission, in line with responsible disclosure principles. All experiments were conducted in a fully controlled local environment, without interacting with any publicly accessible REST APIs, ensuring that no production systems and users were impacted during our evaluation.

5.6 Discussion

Input Generation Constraints Although LLMs are capable of understanding complex parameter constraints described in natural language, we chose to rely on traditional random input generators for efficiency. LLM-driven value generation, when invoked frequently across multiple parameters and test cases, introduces substantial computational overhead and latency. While alternatives such as one-time value pattern generation or sampling from LLM-prepared candidate sets are feasible, we observed that such outputs often overfit to specific endpoints and lack general applicability across the broader API schema. Therefore, we opted not to use an LLM for this step, utilizing a well-established method for generating inputs instead.

Response Analysis Tradeoffs We also avoided using LLMs for interpreting API responses. While LLMs are semantically capable of recognizing nuanced attack indicators, they can also produce high false positive rates due to over interpretation of benign responses. Moreover, the cumulative latency introduced by prompting LLMs at each response handling step would considerably degrade fuzzing throughput. Thus, we rely on traditional oracles for detecting vulnerabilities, to not unnecessarily increase the false positive rate and yet maintain good performance range.

Reduction of FPs One could explore several complementary strategies to reduce false positives in LLM-based API fuzzing further. One direction is the development of more sophisticated heuristic filters that more effectively differentiate benign inconsistencies from genuine security issues. Another promising avenue is the adoption of multi-model verification pipelines in which a second LLM independently evaluates suspected findings and challenges the primary model's reasoning. Additionally, future systems might integrate static or dynamic analysis to trace execution paths or state transitions associated with candidate vulnerabilities, enabling more reliable confirmation and reducing false positives. Together, these enhancements could substantially improve the precision and robustness of LLM-driven fuzzing frameworks. Additionally, a discussion of context size limitations is provided in Appendix C.

6 Related Work

This section provides an overview of related literature in three categories: General REST API Fuzzing, REST API Security Fuzzing, and LLMs in Fuzzing.

General REST API Fuzzing A number of testing tools have been proposed to assess the functional robustness of REST APIs. These primarily operate in a black-box manner, parsing OpenAPI specifications to generate sequences of requests. RESTler [12] is a prominent

Table 7: Detected REST API vulnerabilities across LLM Configurations. A ✓ indicates that the tool successfully identified the vulnerability, while a ✗ denotes failure to detect it.

Application	Endpoint	Vulnerability Type	No LLM	LLaMA 3.3	GPT-4o Mini	DeepSeek-V3	Claude 3.7 Sonnet
Appwrite	GET /avatars/favicon	SSRF	✓	✓	✓	✓	✓
Appwrite	GET /avatars/image	SSRF	✓	✓	✓	✓	✓
Audiobookshelf	POST /api/podcasts/feed	SSRF	✗	✓	✓	✓	✓
Commafeed	POST /rest/feed/fetch	SSRF	✓	✓	✓	✓	✓
Commafeed	GET /rest/feed/subscribe	SSRF	✓	✓	✓	✓	✓
Jellyfin	GET /Images/Remote	SSRF	✓	✓	✓	✓	✓
Jellyfin	GET /Items/RemoteSearch/Image	SSRF	✓	✓	✓	✓	✓
Kavita	POST /api/Theme/download-theme	SSRF	✓	✓	✓	✓	✓
Kavita	POST /api/Upload/upload-by-url	SSRF	✗	✓	✓	✓	✓
Rbaskets	POST /baskets/name	SSRF	✗	✓	✗	✗	✓
Rbaskets	POST /api/baskets/name	SSRF	✗	✗	✗	✗	✓

stateful fuzzer that automatically generates test sequences by inferring dependencies between API endpoints. It continuously refines its understanding of the API's internal state through feedback from prior requests. RestTestGen [58] and RESTest [34] analyze parameter and operation dependencies to systematically construct valid request sequences using constraint solving. MINER [33] applies a neural network to predict critical input parameters, enhancing test generation for complex sequences.

Morest [31] introduces a dynamic RESTful service Property Graph (RPG) to model relationships among endpoints, guiding adaptive test sequence generation. Similarly, other tools [24, 57] have explored grammar-based and coverage-guided techniques to improve parameter fuzzing.

While these tools contribute significantly to API reliability testing, their primary focus remains on maximizing functional coverage and uncovering crashes (e.g., HTTP 500 errors), rather than pinpointing security vulnerabilities.

REST API Security Fuzzing Several tools have emerged to address the specific need for uncovering vulnerabilities in REST APIs. ZAP [46] and Burp Suite are widely used for penetration testing and include extensions to support API specifications. Astra [27] injects malicious payloads into API parameters and headers to detect common web vulnerabilities like SQLi and XSS.

However, these tools typically operate at the level of individual endpoints and cannot reason about multi-endpoint state transitions. NAUTILUS [18] enhances test generation by using policy annotations in API documentation, enabling more realistic operation sequences, though it still focuses mainly on injection flaws.

VoAPI^2 [22] is a recent vulnerability-oriented testing framework that aligns fuzzing strategies with semantic functionality extracted from API specifications. By targeting potentially risky operations, VoAPI^2 effectively narrows the test space and improves vulnerability discovery efficiency.

LLMs in Fuzzing Recent advances have demonstrated the effectiveness of LLMs in fuzzing domains beyond traditional APIs. TitanFuzz [19] and FuzzGPT [20] utilize generative LLMs to synthesize programs that exercise deep learning libraries, achieving high code coverage and bug discovery rates. Similarly, ChatFuzz [50] applies reinforcement learning with LLM-generated sequences to improve hardware verification.

In the domain of smart contracts, LLM4Fuzz [52] uses LLMs to estimate code complexity and vulnerability likelihood, guiding fuzzers toward high-risk regions. ChatAFL [35] focuses on stateful protocol fuzzing by extracting grammars and state transitions using LLMs, enabling deeper coverage and the discovery of unknown bugs.

LogiAgent [60], AutoRestTest [54], and LlamaRestTest [29] have explored the application of LLMs specifically for REST APIs. LogiAgent employs a multi-agent framework to validate business logic constraints, while AutoRestTest and LlamaRestTest integrate LLMs with reinforcement learning for value generation and request adaptation. However, these methods primarily target logical consistency or general testing and are not explicitly focused on uncovering security vulnerabilities.

Our work differs from these approaches by using LLMs explicitly for security vulnerability discovery in REST APIs. By leveraging the contextual reasoning capabilities of LLMs, our fuzzer can identify vulnerabilities arising from multi-endpoint dependencies, implicit authorization patterns, and contextual parameter misuse—areas typically missed by traditional and LLM-based functional fuzzers.

In summary, while significant research has been carried out in API fuzzing and the use of LLMs in testing, few works focus on security-centric REST API fuzzing using LLMs. Our framework is the first to unify stateful request generation, semantic API understanding, and LLM-driven inference with the specific goal of discovering vulnerabilities in real-world REST APIs.

7 Conclusion

In conclusion, RESTing-LLAMA offers a practical and effective approach to fuzzing REST APIs by combining the semantic reasoning capabilities of LLMs with an end-to-end automated pipeline. By operating in a black-box setting and leveraging API documentation, RESTing-LLAMA generates high quality request sequences that uncover both known and novel vulnerabilities. Our evaluation across real-world APIs demonstrates that RESTing-LLAMA significantly reduces false positives and input overhead, highlighting the promise of LLM-guided fuzzing in modern API security testing.

References

[1] [n. d.]. Audiobookshelf. https://github.com/advplyr/audiobookshelf.
[2] [n. d.]. Casdoor. https://github.com/casdoor/casdoor.
[3] [n. d.]. Chatwoot. https://github.com/chatwoot/chatwoot.
[4] [n. d.]. CommaFeed. https://github.com/athou/commafeed.
[5] [n. d.]. Ever Traduora. https://github.com/ever-co/ever-traduora.
[6] [n. d.]. Gitea. https://github.com/go-gitea/gitea.
[7] [n. d.]. Jellyfin. https://github.com/jellyfin/jellyfin.
[8] Analyzing Alpha. 2023. API Statistics: The Growing API Economy. https://analyzingalpha.com/api-statistics
[9] Anthropic. 2025. Claude Sonnet. https://www.anthropic.com/.
[10] Apache Software Foundation. [n. d.]. Answer. https://github.com/apache/answer.
[11] Appwrite Team. [n. d.]. Appwrite: Build like a team of hundreds. https://github.com/appwrite/appwrite.
[12] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RESTler: Stateful REST API Fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*. IEEE Press, Montreal, QC, Canada, 748–758. doi:10.1109/ICSE.2019.00083
[13] Robert Auger. [n. d.]. Path Traversal. http://www.webappsec.org/projects/threat/classes/path_traversal.shtml.
[14] Bernd Bestel. [n. d.]. Grocy. https://github.com/grocy/grocy.
[15] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *International Conference on Software Engineering (ICSE)*.
[16] darklynx. [n. d.]. Request Baskets. https://github.com/darklynx/request-baskets.
[17] DeepSeek-AI et al. 2024. DeepSeek-V3 Technical Report. https://arxiv.org/abs/2412.19437.
[18] Gelei Deng, Zhiyi Zhang, Yuekang Li, Yi Liu, Tianwei Zhang, Yang Liu, Guo Yu, and Dongjin Wang. 2023. NAUTILUS: Automated RESTful API Vulnerability Detection. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5593–5609. https://www.usenix.org/conference/usenixsecurity23/presentation/deng-gelei
[19] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 423–435. doi:10.1145/3597926.3598067
[20] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large Language Models are Edge-Case Fuzzers: Testing Deep Learning Libraries via FuzzGPT. arXiv:2304.02014 [cs.SE] https://arxiv.org/abs/2304.02014
[21] W. Du et al. 2024. VoAPI2: Vulnerability-Oriented API Inspection Framework. https://github.com/NSSL-SJTU/VoAPI2.
[22] Wenlong Du, Jian Li, Yanhao Wang, Libo Chen, Ruijie Zhao, Junmin Zhu, Zhengguang Han, Yijun Wang, and Zhi Xue. 2024. Vulnerability-oriented Testing for RESTful APIs. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 739–755. https://www.usenix.org/conference/usenixsecurity24/presentation/du
[23] Eoftedal, Erez Yalon, and Kingthorin. [n. d.]. Server Side Request Forgery (SSRF). https://owasp.org/www-community/attacks/Server_Side_Request_Forgery.
[24] Patrice Godefroid, Byoungyoung Lee, and Michal Polishchuk. 2020. Intelligent REST API Data Fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 725–736. doi:10.1145/3368089.3409719
[25] Gotson. [n. d.]. Komga. https://github.com/gotson/komga.
[26] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33 (09 2024). doi:10.1145/3695988
[27] Flipkart Incubator. [n. d.]. Astra: Automated Security Testing for REST APIs. https://github.com/flipkart-incubator/Astra.
[28] Kareadita. [n. d.]. Kavita. https://github.com/Kareadita/Kavita.
[29] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. LlamaRestTest: Effective REST API Testing with Small Language Models. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE022 (June 2025), 24 pages. doi:10.1145/3715737
[30] KirstenS, Jim Manico, Jeff Williams, Dave Wichers, Adar Weidman, Roman, Alan Jex, Andrew Smith, Jeff Knutson, Imifos, Erez Yalon, Kingthorin, Vikas Khanna, and Grant Ongers. [n. d.]. Cross Site Scripting (XSS). https://owasp.org/www-community/attacks/xss/.
[31] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: model-based RESTful API testing with execution feedback. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1406–1417. doi:10.1145/3510003.3510133
[32] Sebastian Lubos, Alexander Felfernig, Thi Ngoc Trang Tran, Damian Garber, Merfat El Mansi, Seda Polat Erdeniz, and Viet-Man Le. 2024. Leveraging LLMs for the Quality Assurance of Software Requirements. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 389–397. doi:10.1109/re59067.2024.00046
[33] Chenyang Lyu, Jiacheng Xu, Shouling Ji, Xuhong Zhang, Qinying Wang, Binbin Zhao, Gaoning Pan, Wei Cao, Peng Chen, and Raheem Beyah. 2023. MINER: A Hybrid Data-Driven Approach for REST API Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 4517–4534. https://www.usenix.org/conference/usenixsecurity23/presentation/lyu
[34] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2020. RESTest: Black-Box Constraint-Based Testing of RESTful Web APIs. In *Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings* (Dubai, United Arab Emirates). Springer-Verlag, Berlin, Heidelberg, 459–475. doi:10.1007/978-3-030-65310-1_33
[35] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large Language Model guided Protocol Fuzzing. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society. https://www.ndss-symposium.org/ndss-paper/large-language-model-guided-protocol-fuzzing/
[36] Microcks.io. [n. d.]. Microcks. https://github.com/microcks/microcks.
[37] Microsoft. [n. d.]. RESTler Fuzzer – Compiler Source Code. https://github.com/microsoft/restler-fuzzer/tree/main/src/compiler.
[38] D. Molteni. 2021. Landscape of API Traffic. https://blog.cloudflare.com/landscape-of-api-traffic.
[39] Fatemeh N. Motlagh, Mahdi Hajizadeh, Mahan Majd, Parham Najafi, Feiyu Cheng, and Christoph Meinel. 2024. Large Language Models in Cybersecurity: State-of-the-Art. https://arxiv.org/abs/2402.00891. arXiv:2402.00891.
[40] National Institute of Standards and Technology. 2023. CVE-2023-27163 Detail. https://nvd.nist.gov/vuln/detail/CVE-2023-27163
[41] National Vulnerability Database. 2018. CVE-2018-15192: SSRF in Gitea and Gogs Webhooks. https://nvd.nist.gov/vuln/detail/CVE-2018-15192.
[42] National Vulnerability Database. 2021. CVE-2021-29490: Jellyfin SSRF via imageUrl Parameter. https://nvd.nist.gov/vuln/detail/CVE-2021-29490.
[43] National Vulnerability Database. 2023. CVE-2023-27159: Appwrite SSRF via /v1/avatars/favicon. https://nvd.nist.gov/vuln/detail/CVE-2023-27159.
[44] OpenAI. 2025. GPT-4o Mini. https://openai.com/.
[45] Outline. [n. d.]. Outline. https://github.com/outline/outline.
[46] OWASP. [n. d.]. OWASP Zed Attack Proxy (ZAP). https://www.zaproxy.org/.
[47] OWASP. 2023. API Security Top 10. https://owasp.org/www-project-api-security/.
[48] Henry Pearce, Ben Tan, Bilal Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining Zero-Shot Vulnerability Repair with Large Language Models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2339–2356. doi:10.1109/SP46215.2023.10179324
[49] Postman Inc. 2024. 2024 State of the API Report. https://www.postman.com/state-of-api/.
[50] Mohamadreza Rostami, Marco Chilese, Shaza Zeitouni, Rahul Kande, Jeyavijayan Rajendran, and Ahmad-Reza Sadeghi. 2024. Beyond Random Inputs: A Novel ML-Based Hardware Fuzzing. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. doi:10.23919/DATE58400.2024.10546625
[51] Salt Security. 2025. Salt Security State of API Security Report. https://content.salt.security/state-api-report.html.
[52] Chaofan Shou, Jing Liu, Doudou Lu, and Koushik Sen. 2024. LLM4Fuzz: Guided Fuzzing of Smart Contracts with Large Language Models. arXiv:2401.11108 [cs.CR] https://arxiv.org/abs/2401.11108
[53] Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. AutoRestTest: A Tool for Automated REST API Testing Using LLMs and MARL. arXiv:2501.08600 [cs.SE] https://arxiv.org/abs/2501.08600
[54] Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. AutoRestTest: A Tool for Automated REST API Testing Using LLMs and MARL. *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)* (2025), 21–24. https://api.semanticscholar.org/CorpusID:275544555
[55] The Linux Foundation. 2024. OpenAPI Specification v3.1.0. https://spec.openapis.org/oas/latest.html.
[56] Hugo Touvron et al. 2025. LLaMA 3.3: Open Foundation and Fine-Tuned Chat Models. https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct.
[57] Chung-Hsuan Tsai, Shi-Chun Tsai, and Shih-Kun Huang. 2021. REST API Fuzzing by Coverage Level Guided Blackbox Testing. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. 291–300. doi:10.1109/QRS54544.2021.00040
[58] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTESTGEN: Automated Black-Box Testing of RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 142–152. doi:10.1109/ICST46399.2020.00024
[59] Wikipedia contributors. 2025. Representational State Transfer. https://en.wikipedia.org/wiki/Representational_state_transfer.

Table 8: Comparison of Baselines on Equal Budget

Tool	Avg. Packets	Avg. Time
RESTler	52,691	52m 01s
MINER	91,770	60m 00s
RestTestGen	29,458	3m 47s
VOAPI2	465	2m 26s
RESTing-LLAMA	1,669	8m 14s

[60] Ke Zhang, Chenxi Zhang, Chong Wang, Chi Zhang, YaChen Wu, Zhenchang Xing, Yang Liu, Qingshan Li, and Xin Peng. 2025. LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents. *ArXiv* abs/2503.15079 (2025). https://api.semanticscholar.org/CorpusID:277113189

[61] Weilin Zhong, Wichers, Amwestgate, Rezos, Clow808, KristenS, Jason Li, Andrew Smith, Jmanico, Tal Mel, and Kingthorin. [n. d.]. Command Injection. https://owasp.org/www-community/attacks/Command_Injection.

A Prompt Templates

Figure 8 presents a sample prompt template carefully crafted for the dependency analysis step of our RESTing-LLAMA fuzzer. Sample prompt templates for other steps of the fuzzer will be provided with artifacts.

B Basline Comparison on Equal Budget

Table 8 presents a fair, equal-budget comparison in which every fuzzer was allocated the same one-hour time window per target and conducted two independent trails. The experiments were eonducted on identical hardware Ubuntu 22.04 OS, 32 CPU cores, and 256 GB RAM. Under identical constraints, RESTler and MINER consistently produced the largest request volumes, while RESTing-LLAMA generated two to three orders of magnitude fewer requests across most applications. This leads to markedly higher throughput efficiency: for example, on average on all targets, RESTler issued 52k requests in one hour (~14.6 req/s), whereas RESTing-LLAMA exercised only 1k requests (~0.46 req/s) yet still validated all target vulnerabilities. This demonstrates that each request generated by RESTing-LLAMA is more meaningful, allowing it to uncover vulnerabilities without sending excessive traffic. These runs were used to compare throughput characteristics and were *not* checked for vulnerabilities.

Finally, the LLM cost of RESTing-LLAMA remains modest. Across all six benchmark targets, RESTing-LLAMA with its six-stage fuzzing pipeline and multiple vulnerability-specific prompts, yielding an average inference cost of approximately 1 per target when executed with Claude Sonnet 4.5. This total cost for each target is comparably convincing to traditional fuzzers' compute cycles, while providing substantial semantic guidance that improves precision and reduces wasteful request generation.

C Discussion

Context Limitations Another practical limitation stems from the context window of available LLM APIs. During evaluation, several real-world API specifications exceeded the maximum token limits particularly for OpenAI and DeepSeek APIs (128K tokens), and more so for local models like LLaMA (20K tokens). Only Claude 3.7 Sonnet (200K context) could consistently accommodate full documentation without truncation. This limitation impacts the effectiveness of LLM reasoning, especially when context fragmentation leads to omitted dependency details. As future work, integrating prompt partitioning or retrieval-augmented generation (RAG) methods may help overcome these bottlenecks while preserving performance.

D Detection Strategies of Vulnerabilities

D.1 Cross-Site Scripting

Cross-Site Scripting (XSS) vulnerabilities allow attackers to inject malicious JavaScript code into web applications, which is then executed in the context of another user's browser. This can lead to various security risks, such as session hijacking.XSS typically arises from improper sanitization of input data before it is rendered by the browser. Common variants include reflected XSS, stored XSS, and DOM-based XSS. To successfully exploit an XSS vulnerability, certain conditions must be met. One important factor is the Content-Type response header, which must be set to text/html or an equivalent. In REST APIs, however, this header is usually set to application/json, as responses typically return JSON data. This setting is often configured globally to enforce JSON communication, making REST APIs less susceptible to traditional XSS attacks by design.

Nevertheless, the fuzzer actively searches for XSS vulnerabilities. Figure 9 illustrates the XSS detection process. To detect XSS vulnerabilities, we analyze server responses for injected payloads. For detecting reflected XSS, we try to inject payloads using request parameters and inspect the resulting HTML output. If the response contains our payload and is served with a Content-Type of text/html, we detected a reflected XSS vulnerability successfully. In Figure 9, this is illustrated by the fuzzer receiving a direct response from the REST API.

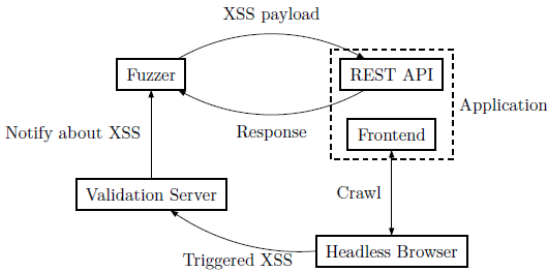


Figure 9: Sequence diagram of the XSS detection process

In addition, the fuzzer tests for stored XSS by continuously crawling the application using a headless browser. This browser visits and renders all accessible pages, checking for the execution of previously injected JavaScript payloads. If a payload executes, the script contacts the same external validation server used in SSRF detection. The server then reports back to the fuzzer, which identifies the triggering test case based on the unique test case ID embedded in the payload.

Although these kinds of XSS vulnerabilities are fundamentally a client-side issue, some REST API fuzzers, like VoAPI²[22], still consider them a valid vulnerability. VoAPI² in particular use a manual approach for checking whether a stored XSS vulnerability was triggered. Our approach partly automates this process by triggering these vulnerabilities while crawling the application.

Dependency Analysis Prompt Template:
Here is a REST API documentation:

```
{{ range $endpoint := .Docs.Endpoints }}
- endpoint: "{{ $endpoint.Method }} {{ $endpoint.Path }}"
  description: "{{ $endpoint.Description | replace "\n" " " | trim }}"
  {{- if $endpoint.RequestParameters }}
  request_parameters:
    {{- range $parameter := $endpoint.RequestParameters }}
    - {{ $parameter.Name }}
    {{- end }}
  {{- else }}
  request_parameters: []
  {{- end }}
  {{- if $endpoint.ResponseParameters }}
  response_parameters:
    {{- range $parameter := $endpoint.ResponseParameters }}
    - {{ $parameter.Name }}
    {{- end }}
  {{- else }}
  response_parameters: []
  {{- end }}
{{- end }}
```

Your task is to analyze this documentation and identify all dependencies between the API endpoints. An endpoint is dependent on another endpoint if the latter produces a value that the first endpoint requires as an input to function correctly.
List each endpoint with its dependencies, specifically the endpoint and parameter it depends on. Ensure that no dependency is left out.
This information will be used to prepare the API for fuzzing, so please ensure all dependencies are fully and accurately captured.
The output MUST be in JSON format.
The JSON output MUST be structured like in this example:

```
{
  "DELETE /users/{id}": [
    {
      "endpoint": "POST /users",
      "producer": "user_id",
      "consumer": "id"
    }
  ],
  "GET /users/self": []
}
```

The producer is the response parameter the endpoint depends on. The consumer is the request parameter of the endpoint, which needs the response parameter.
Any parameters used for authentication MUST be ignored. Make sure the producer and consumer keys are exact. Do not add any more text to them.
Furthermore, only output this JSON object and nothing else.

Figure 8: Prompt template used for dependency analysis of REST API documentation.

D.2 Command Injection

Command Injection vulnerabilities occur when an application uses user input improperly and allows the execution of system commands by an attacker. This flaw is exploited by injecting commands into request parameters that are later used in shell commands by the server.

For Command Injection vulnerabilities to be exploitable, the application must embed user input into shell commands without proper validation and sanitization. This usually happens when such input is directly used as part of a shell command. Therefore, Command Injection requires strict validation and sanitization of input.

There are multiple methods of testing Command Injection vulnerabilities. Often, a time-based method is used, where the fuzzer tries to inject a sleep command into the underlying shell. The Command Injection vulnerability is considered present when a noticeable delay is detected. The delay should have a similar length to the wait time provided to the sleep command. While this method works, it can be prone to false positives. If the application experiences a heavy load, for example, this time-based approach can be unreliable, as a delay in the response time could be linked to the unavailability of system resources to process the request in an adequate amount of time.

Our RESTing-LLAMA, uses another method for detection. We detect Command Injection vulnerabilities by attempting to inject commands that send an HTTP request to our validation server. Again, this server is the same one used for detecting other vulnerabilities like SSRF. If the fuzzer successfully injects a command that executes a curl command, sending a request to our validation server, the server will notify the fuzzer, similar to the other vulnerabilities, confirming the vulnerability.

This method is less susceptible to false positives in scenarios with limited resources, as the fuzzer receives a clear signal when the vulnerability is triggered. However, our detection methods rely on the presence of the curl command on the targeted system. Although curl is a very wide-spread program often pre-installed on Linux distributions, some popular minimal docker images, like Alpine [88], do not have it pre-installed. Nonetheless, our method reduces the amount of false-positives.

D.3 Path Traversal

Path Traversal vulnerabilities allow an attacker to manipulate file paths on the server that use input parameters. Again, these attacks exploit the missing validation of input data. This allows referencing files outside the intended directories by using sequences like .. or absolute paths. Attackers often target system files like /etc/passwd on Unix-based systems, revealing user authentication data on the server.

To prevent Path Traversal vulnerabilities, server-side applications must validate inputs to file paths. File directory white lists should be used to ensure that the resulting paths only point to files located in an intended directory. Modern frameworks for web applications provide built-in tools to handle path inputs from users by abstracting away any details associated with the file system directly. Developers should use such tools instead of implementing their own solution.

The fuzzer uses payloads that try to retrieve content from widespread system files, like / etc/passwd on Unix systems. Regular expressions are used to identify response data that looks like any of these files. If any of the received responses contains such a pattern a Path Traversal vulnerability is recorded. While this process can result in false-positives, if API responses by chance include matching text patterns, this should be a rare occurrence, as the contents of these files are a lot different from the JSON format expected from REST API responses. Ultimately, our Path Traversal detection approach is an effective identification method with only a small chance of encountering false positives.